CampusHub 项目“架构辩论赛”记录

1. 活动基本信息

项目	内容
项目名称	校园互助服务平台 CampusHub
阶段	Phase 2 体系结构设计
活动名称	架构辩论赛
活动时间	2026 年 4 月 28 日
参与成员	吴禹成（成员 A）、徐航宇（成员 B）、李宇瀚（成员 C）、管泽昊（成员 D）
活动目标	比较不同 Prompt 策略下 AI 给出的架构建议质量，并据此确定项目最终架构方案

说明：以下 AI 回答为归档整理版，保留了原始建议的核心结论、关键论据与风险提示，便于直接纳入课程文档。

2. 项目约束共识

在正式进行“架构辩论赛”前，团队先统一了本项目的约束条件，作为评估 AI 回答质量的标准：

1. 团队规模为 4 人，开发周期共 10 周，P2 阶段必须优先服务于 P0 功能落地。
2. P0 必做功能包括用户管理、需求发布与浏览、订单流程管理。
3. P1 应做功能包括评价与信用体系、站内消息通知。
4. P2 可选功能包括智能匹配与推荐、管理后台、即时通讯。
5. 非功能需求明确提出：核心页面加载时间小于 2 秒，具备基本安全防护能力，界面简洁，代码结构清晰，关键模块需要具备可测试性。
6. 团队更适合采用实现和部署复杂度较低、但后续又有一定演进空间的架构方案。

3. 四位成员的 Prompt 与 AI 回答记录

3.1 成员 A：直接询问“推荐什么架构”

Prompt

```
我们要做一个校园互助服务平台 CampusHub，包含用户管理、需求发布、接单、订单状态流转、评价、消息通知这些功能。你推荐我们采用什么
```

AI 回答（整理版）

1. 推荐采用传统 MVC 或三层架构的单体应用。
2. 理由是该系统核心流程以业务表单、列表查询、状态流转和基础权限控制为主，属于典型的 Web 管理与交易型系统，使用单体架构开发速度更快。
3. 建议按功能划分控制器、服务层和数据访问层，前端负责页面展示，后端提供接口。

4. 可优先建设用户模块、需求模块、订单模块、评价模块、消息模块和管理模块。
5. 技术上可使用 Vue 或 React 作为前端，后端使用 Spring Boot 或 Node.js，数据库使用 MySQL。
6. 不建议一开始就使用微服务，因为系统规模较小，会增加部署和联调成本。

团队评价

1. 回答速度快，能迅速给出方向。
2. 结论基本合理，但较为泛化，没有充分考虑“4 人团队、10 周周期、P0/P1/P2 分层目标”这些课程约束。
3. 对“为什么不选其他方案”的论证不够充分，质量属性权衡也较少。

3.2 成员 B：提供详细项目约束后再询问

Prompt

```
请基于以下约束为 CampusHub 推荐架构方案：
1. 团队 4 人，开发周期 10 周；
2. P0 必须完成用户管理、需求发布与浏览、订单流程管理；
3. P1 要完成评价与信用体系、站内通知；
4. P2 可选智能推荐、管理后台、即时通讯；
5. 核心页面加载时间要小于 2 秒；
6. 团队运维经验有限，希望开发、测试、部署尽量简单；
7. 需要保证后续功能还能继续扩展。

请给出：
1. 最适合的架构；
2. 主要模块划分；
3. 是否需要缓存或消息机制；
4. 不推荐采用什么方案，并说明原因。
```

AI 回答（整理版）

1. 最适合采用“前后端分离 + 分层的模块化单体架构”。
2. 前端与后端分离后，前端可独立迭代页面与交互，后端专注业务逻辑与数据管理，利于 4 人团队并行开发。
3. 后端仍保持单体部署，但内部需要按业务域切分模块，至少包括用户认证、个人资料、需求发布、需求浏览、订单流转、评价信用、消息通知、后台管理等模块。
4. 推荐在后端内部采用 Controller、Service、Repository 三层划分，以降低代码耦合度并提升可测试性。
5. 数据库使用 MySQL 即可满足初期需求；缓存不是首要必需项，但可在需求列表、热门分类和登录态管理中引入 Redis 作为可选优化。
6. 消息机制不建议在一开始建设完整消息队列系统，但可以在应用内部预留异步处理能力，用于站内通知、评价提醒等非强一致场景。
7. 明确不推荐微服务架构，因为服务拆分、接口治理、部署监控、日志追踪和权限管理都会显著增加团队负担；也不推荐纯事件驱动架构作为主架构，因为其调试复杂，数据一致性与排错成本较高。

团队评价

1. 回答明显比成员 A 的结果更贴合项目现实，尤其是把“单体部署、模块化设计、局部引入异步机制”区分得比较清楚。
2. 对“为什么不选其他方案”的解释更具体，可直接服务于后续 ADR 编写。
3. 缺点是对未来演进路径说明较少，例如项目规模扩大后如何拆分服务、如何演进通知系统等内容讲得不够深入。

3.3 成员 C：要求 AI 对比多种架构方案优劣

Prompt

```
请对以下几种架构在校园互助服务平台 CampusHub 中的适用性做对比：
1. MVC / 分层单体架构
2. 微服务架构
3. 事件驱动架构

请至少从开发复杂度、可维护性、可扩展性、性能、团队学习成本、10 周内可行性六个维度进行比较，并在最后给出推荐方案和不推荐方案。
```

AI 回答（整理版）

对比维度	MVC / 分层单体	微服务	事件驱动
开发复杂度	低，便于快速启动	高，需要服务拆分、 治理与部署体系	中高， 需要事件模型设计与异步处理机制
可维护性	中到高， 前提是模块边界清晰	高， 但前提是团队具备工程治理能力	中，解耦较好，但排错链路复杂
可扩展性	中， 适合早期和中小规模	高，适合高并发和多团队协作	高，适合通知、推荐、日志等异步场景
性能	对当前规模足够	理论上最好， 但收益在本项目中不明显	对异步吞吐有优势， 但核心交易链路不一定更优
团队学习成本	低	高	中高
10 周内可行性	高	低	中

AI 最终给出的建议是：

1. 主架构应选择分层的模块化单体架构，而不是微服务或纯事件驱动。
2. 如果需要保留后续扩展空间，可以在通知和推荐相关模块中使用“内部事件”或“异步任务”的思想，但不应把整个系统设计成完全事件驱动。
3. 对于课程项目而言，最重要的不是理论上的最大扩展性，而是按时交付 P0 功能并保持代码结构清晰。

团队评价

1. 这轮回答的最大价值在于“横向比较”做得很清楚，适合直接放入候选方案分析表。
2. AI 明确指出了“理论最优”和“课程项目最优”并不相同，这一点对团队形成统一认识很有帮助。
3. 缺点是回答偏分析型，落地性略弱，模块接口和演进步骤没有成员 B、成员 D 那么具体。

3.4 成员 D：要求 AI 扮演架构师角色进行分析

Prompt

```
请扮演一名有高校互联网平台经验的软件架构师，为 CampusHub 设计一套“10 周内能落地、后续还能演进”的架构方案。请重点回答：
1. 主架构风格应该是什么；
2. 模块如何划分，边界如何控制；
3. 哪些地方需要同步调用，哪些地方适合异步处理；
4. 主要质量属性如何权衡；
5. 如果未来用户量增长，再如何从当前架构演进。
```

AI 回答（整理版）

1. 主架构建议采用“前后端分离 + 模块化单体 + 分层架构”，并在单体内部引入轻量的领域边界意识。
2. 系统核心模块应划分为：用户与认证、个人资料、需求管理、订单管理、评价与信用、消息通知、后台管理、推荐扩展模块。
3. 同步调用应保留在强一致业务链路中，例如注册登录、发布需求、接单、订单状态变更、评价提交等流程；异步处理适合用于站内通知、评价提醒、推荐计算、运营统计等场景。
4. 质量属性权衡上，本项目应优先保证“开发效率、可维护性、可测试性、简单部署”，而不是一开始追求“极致可扩展性”。
5. 单体内部必须通过模块边界、接口抽象和统一 DTO 限制跨模块直接依赖，避免后期变成“大泥球”。
6. 如果后续项目规模增长，可先把通知、推荐、后台统计这些天然边界清晰、强一致性要求较低的模块优先拆分为独立服务，而不是一开始就把用户、订单、需求等核心交易模块拆开。
7. 风险提示包括：若模块边界失控，单体也会退化；若过早引入消息队列或微服务，会让开发和联调成本超过项目收益；若推荐功能过早复杂化，会压缩 P0 交付资源。

团队评价

1. 这是四轮回答中最完整的一轮，兼顾了当前落地和未来演进。
2. AI 明确区分了同步链路与异步链路，也明确指出未来若要拆分服务，应优先拆分通知、推荐、统计等边界清晰模块，这一建议很有工程可执行性。
3. 缺点是篇幅较长，部分表述偏“架构评审口径”，需要人工压缩成课程文档可直接使用的语言。

4. 不同 Prompt 策略下 AI 回答质量比较

成员	Prompt 策略	回答质量（1-5）	主要优点	主要不足
成员 A	直接提问	3.0	回答快，能快速给出基本方向	过于笼统， 缺少约束条件与权衡分析
成员 B	提供详细项目约束	4.5	贴合课程项目现实， 可直接转化为架构方案	未来演进部分略简单
成员 C	对比多种架构优劣	4.0	横向比较清楚， 适合做候选方案分析	落地细节较少
成员 D	角色扮演为架构师	4.8	权衡完整，风险和演进路径清晰	内容较长，需要人工整理

团队结论

1. Prompt 越具体，AI 给出的方案越贴近项目真实约束。
2. 只问“推荐什么架构”，AI 往往给出正确但较泛化的答案。

3. 当 Prompt 中加入“约束条件”“对比维度”“角色设定”后，AI 回答在可执行性、权衡深度和可复用性上明显提升。
4. 角色扮演类 Prompt 生成的内容最接近 ADR 和架构评审语言，但仍需要人工压缩与筛选，不能直接照抄。

5. 团队辩论纪要

本组围绕四轮 AI 建议进行了约 30 分钟讨论，讨论重点如下。

议题	讨论摘要	最终结论
是否采用微服务	少数成员认为微服务“看起来更先进”，但多数成员指出团队规模小、时间紧、运维能力有限，若一开始采用微服务，将在网关、服务注册、分布式调用、日志追踪和部署上投入过多精力。	不采用微服务作为当前主架构。
是否采用纯事件驱动	团队认可事件驱动在通知、推荐、统计等场景中的价值，但担心其作为主架构会增加调试和一致性处理难度。	不采用纯事件驱动作为主架构，只保留局部异步处理思想。
是否需要前后端分离	考虑到前端页面、交互逻辑和后端业务逻辑都比较明确，前后端分离能提高并行开发效率，也方便后续迭代和接口测试。	采用前后端分离。
单体架构是否会后期失控	团队担心单体后期变成“大泥球”，因此重点讨论了模块边界、接口隔离和统一 DTO 的必要性。	采用“模块化单体”，而不是“无边界单体”。
智能推荐是否进入主链路	团队一致认为智能推荐属于 P2 可选增强功能，不应影响 P0 主流程。	推荐功能单独设计为可插拔扩展模块，不进入首版核心链路。

6. 投票结果与最终架构决议

候选方案	票数	结果
前后端分离 + 分层的模块化单体架构 + 局部异步机制	4	通过
微服务架构	0	未通过
纯事件驱动架构	0	未通过

最终决议

本项目最终采用“前后端分离 + 分层的模块化单体架构”作为主架构风格。系统部署上保持单体，代码组织上按业务域划分模块，核心交易链路采用同步调用，消息通知、评价提醒、推荐计算、统计分析等非强一致场景可采用内部异步任务或轻量事件机制处理。

7. 选择该方案的原因

1. 与团队规模和工期匹配。4 人团队、10 周周期更需要低复杂度、高交付效率的方案，模块化单体明显优于微服务。
2. 能覆盖课程要求中的核心功能。用户管理、需求发布、订单流转、评价信用、消息通知都可以在该架构下清晰建模与实现。
3. 易于测试与部署。单体应用更适合课程项目阶段的联调、问题定位和版本发布。
4. 保留后续演进空间。虽然当前不拆微服务，但通过明确模块边界、统一接口和领域隔离，未来仍可按通知、推荐、统计等方向逐步拆分。
5. 更符合质量属性优先级。本项目优先级不是“无限扩展性”，而是“按时交付、结构清晰、便于维护、性能达标”。

8. 为什么不选其他方案

8.1 不选微服务架构

1. 微服务更适合业务复杂度高、团队人数多、运维能力成熟的场景，与当前课程项目规模不匹配。
2. 微服务会带来网关、服务治理、链路追踪、配置管理、分布式事务等额外复杂度，成本远超收益。
3. 对本项目而言，真正的瓶颈更可能出现在需求分析和业务实现，而不是单体架构本身的扩展能力。

8.2 不选纯事件驱动架构

1. 纯事件驱动更适合大量异步处理和高解耦场景，但 CampusHub 的核心业务是订单状态流转与用户操作确认，主链路更强调清晰性和一致性。
2. 如果一开始全面事件化，会增加系统理解难度、调试难度和错误排查成本。
3. 团队认为事件机制适合做“局部增强”，不适合作为当前主架构。

8.3 不选无模块边界的传统 MVC 单体

1. 虽然简单 MVC 单体也能快速启动，但如果不强调模块边界，后期容易出现控制器臃肿、服务层混杂、跨模块直接依赖等问题。
2. 课程后续阶段还会继续扩展功能，因此需要比“纯粹能跑起来”更高一点的结构化设计。
3. 因此团队采用“模块化单体”而不是“松散单体”。

9. 本次辩论对 AI 使用的反思

1. AI 在“架构方向建议”和“候选方案对比”上非常高效，能快速帮助团队缩小决策范围。
2. AI 的局限主要体现在容易给出“看起来正确但不贴合团队约束”的泛化建议，因此必须提供明确上下文。
3. AI 很擅长生成“原则性答案”，但是否真的适合本项目，仍需要团队结合工期、成员能力、交付风险进行人工判断。
4. 本次最有效的 Prompt 策略是“详细约束 + 指定输出结构”以及“角色扮演 + 要求质量属性权衡”。